

C Programming

6_pointers

```
int scores[3] = {77, 43, 100};  
char ch = 'a';
```

	[0]			[1]	[2]			
Value	...	...	77	43	100	...	'a'	...
Address	0	200	204	208		400	429	4967296

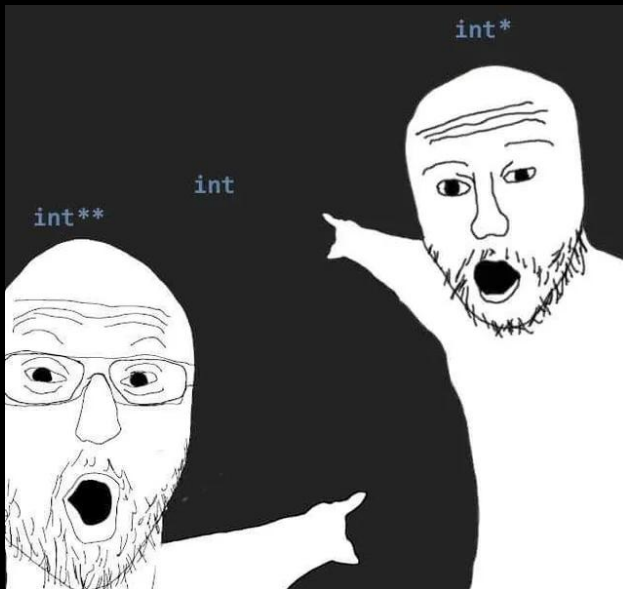
Память похожа на очень длинный массив байт, где индексы этого массива - есть адреса (строка с Address), а значения массива - значения, что хранятся в памяти (строка Value)

`&scores[0], &scores[1], &scores[2]`

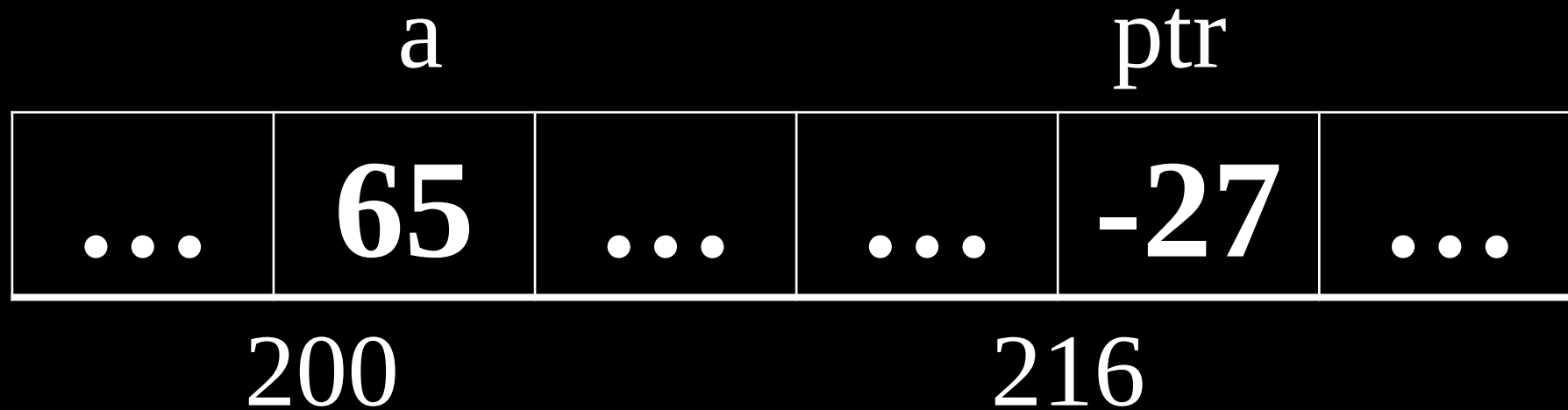
Адрес - номер в памяти с нуля, обозначающийся целым числом. ``&`` - операция взятия адреса переменной

Кстати, `scores == &scores[0]`

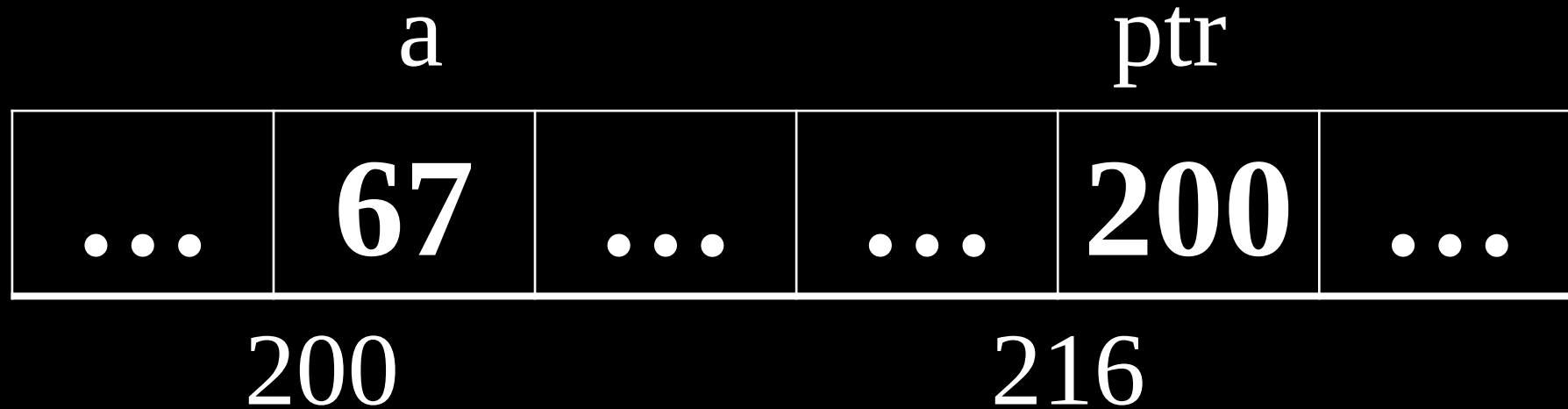
Указатель (pointer) - это переменная, которая хранит адрес ячейки памяти.



```
char a = 'A'; // 65
char *ptr;
```




```
char a = 'A'; // 65
char *ptr;
ptr = &a;
*ptr = 67;
```



Разыменовыванием (dereference) - называется переход по адресу, что хранится в переменной-указателе для считывания или записи значения по указанному адресу

Здесь не путаем `\*` когда объявляем и когда разыменовываем переменную, это разные вещи:

- `int *ptr;` - `\*` здесь значит, что мы объявляем указатель
- `*ptr = 5;` - `\*` здесь означает операцию разыменовывания

При объявлении указателя есть **ТОЛЬКО ДВА** варианта где поставить звездочку:

- `int *ptr;`
- `int* ptr;`

Void pointers

```
void b = 5; // Ошибка!
```

```
void *ptr; // Указатель на пустоту
```

```
sizeof(char *) == sizeof(void *) == sizeof(int *)
```

Для 32-разрядных систем размер указателя будет 4 байта, для 64 = 8 байт.

void указатель нельзя разыменовывать, если явно не преобразовать его на указатель другого типа

```
int a = 5;  
void *ptr = &a;  
*ptr = 3; // ошибка!
```

```
int a = 5;  
void *ptr = &a;
```

```
char b = *((char *)ptr);
```

NULL

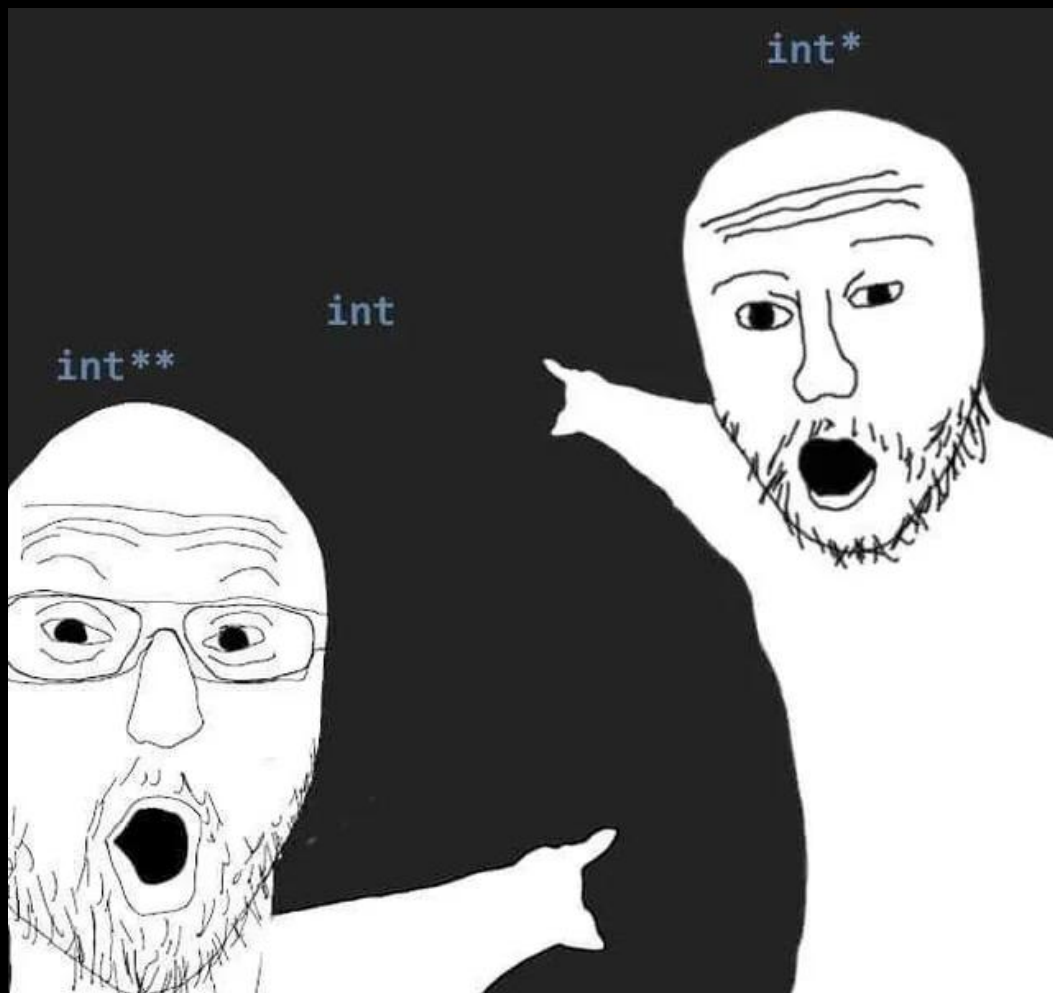
Для указателей есть свой 0 - NULL. NULL - это адрес на нулевую ячейку к памяти. Всем программам к ней доступ закрыт и при попытке разыменовать NULL получим segfault.

```
int *ptr = NULL;  
// ..  
if (ptr != NULL) { // ptr инициализирован?  
    // Что-то выполняем  
}
```

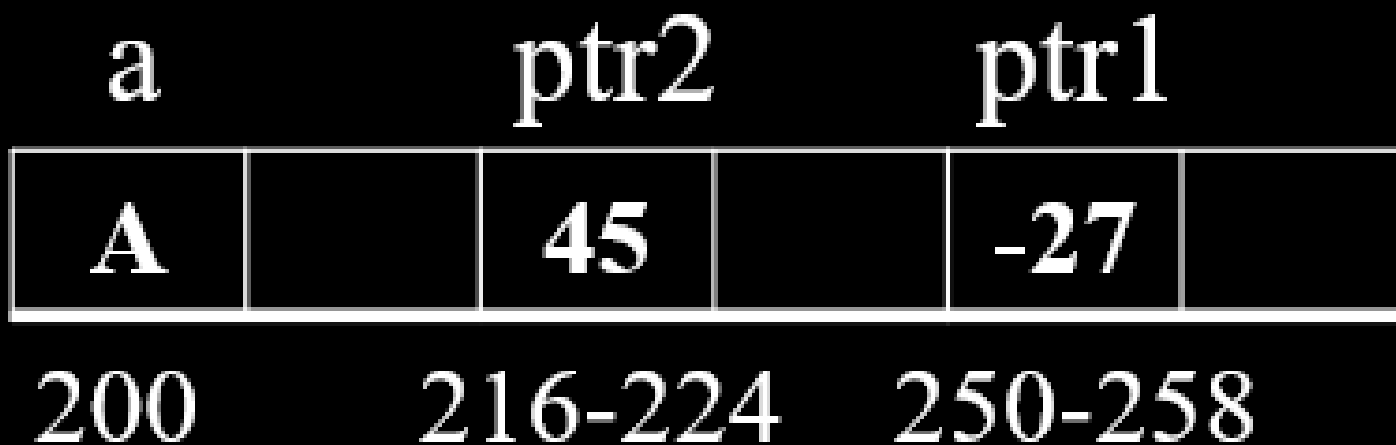
Что здесь объявлено?

```
int* a, b;
```

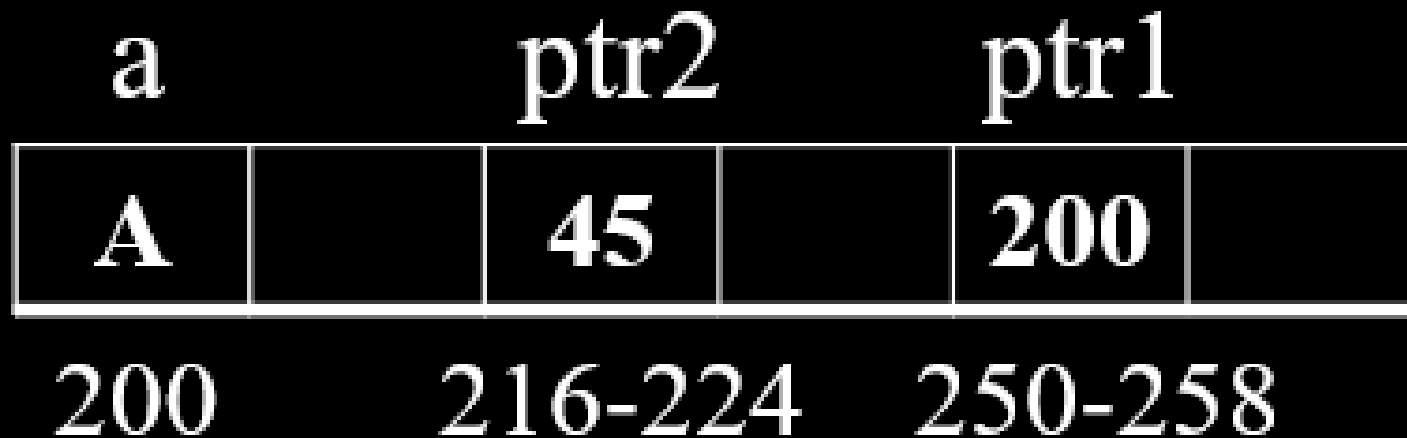
Указатель на указатель на ..



```
char a = 'A';  
char *ptr1;  
char **ptr2; // указатель на указатель на int
```



```
char a = 'A';  
char *ptr1;  
char **ptr2; // указатель на указатель на int  
  
ptr1 = &a;
```



```
char a = 'A';  
char *ptr1;  
char **ptr2; // указатель на указатель на int
```

```
ptr1 = &a;  
ptr2 = &ptr1;
```




```

printf("ptr2 = %p\n", ptr2);           // 250
printf("*ptr2 = %p\n", *ptr2);         // 200
printf("**ptr2 = %c\n", **ptr2);       // A
printf("&ptr = %p\n", &ptr2);          // 216

```

a	ptr2		ptr1	
A		250		200
200	216-224		250-258	

```
int a = 123;  
int *ptr = &a;  
  
*&*&*&*ptr = ?
```

Pointers to pointers

```
char a = 'A';  
char *ptr1;  
char **ptr2;
```

```
ptr1 = &a;
```

```
ptr2 = &ptr1;  
printf("ptr2 = %p\n", ptr2);      // 250  
printf("*ptr2 = %p\n", *ptr2);    // 200  
printf("**ptr2 = %c\n", **ptr2);  // A  
printf("&ptr = %p\n", &ptr2);     // 216
```

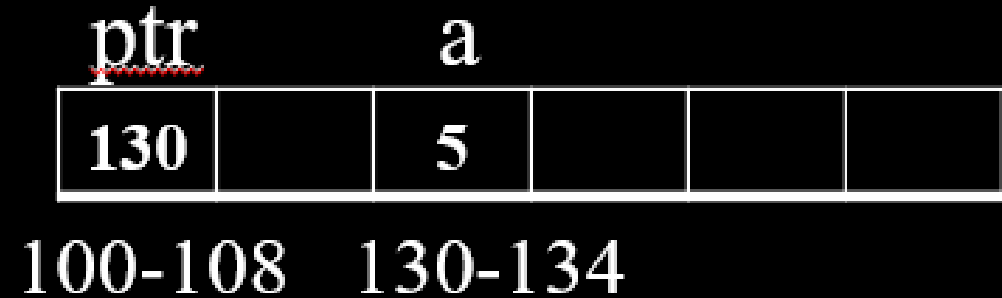
a		ptr2		ptr1	
A		45		-27	
200		216-224		250-258	

a		ptr2		ptr1	
A		45		200	
200		216-224		250-258	

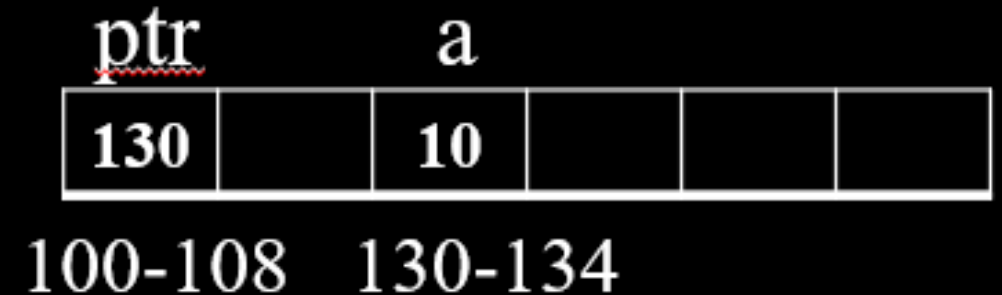
a		ptr2		ptr1	
A		250		200	
200		216-224		250-258	

Pointer arithmetics

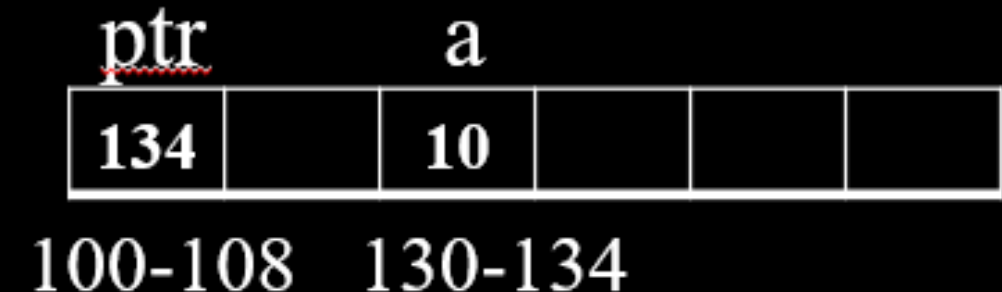
```
int a = 5;  
int *ptr = &a;
```



```
*ptr = *ptr + 5;
```



```
ptr = ptr + 1;
```



```
int arr[] = {10, 20, 30, 40, 50, 60, 70};  
int *ptr = arr;
```

```
printf("%d\n", *ptr);           // = ?  
printf("%d\n", *(ptr + 3));    // = ?  
printf("%d\n", ptr[2]);        // = ?  
printf("%ld\n", (ptr + 4) - ptr); // = ?
```

```
int matrix[3][4] = {  
    {1, 2, 3, 4},  
    {5, 6, 7, 8},  
    {9, 10, 11, 12}  
};
```

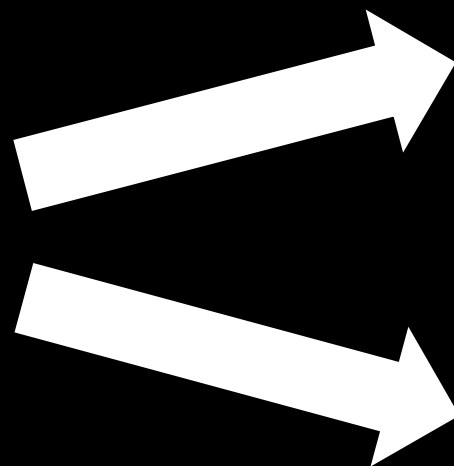
```
int (*ptr)[4] = matrix;
```

```
printf("%d\n", **ptr);           // = ?  
printf("%d\n", *(*ptr + 1) + 2); // = ?  
printf("%d\n", (*ptr)[3]);       // = ?  
printf("%d\n", *(ptr[2] + 1));   // = ?  
printf("%ld\n", (char*)(ptr + 2) - (char*)ptr); // = ?
```

Endianness (порядок байт)

- Big Endian (обратный порядок): Сначала старший байт, число слева направо
- Little Endian (прямой порядок): Сначала младший байт, число справа налево

`int a = 0x12345678`



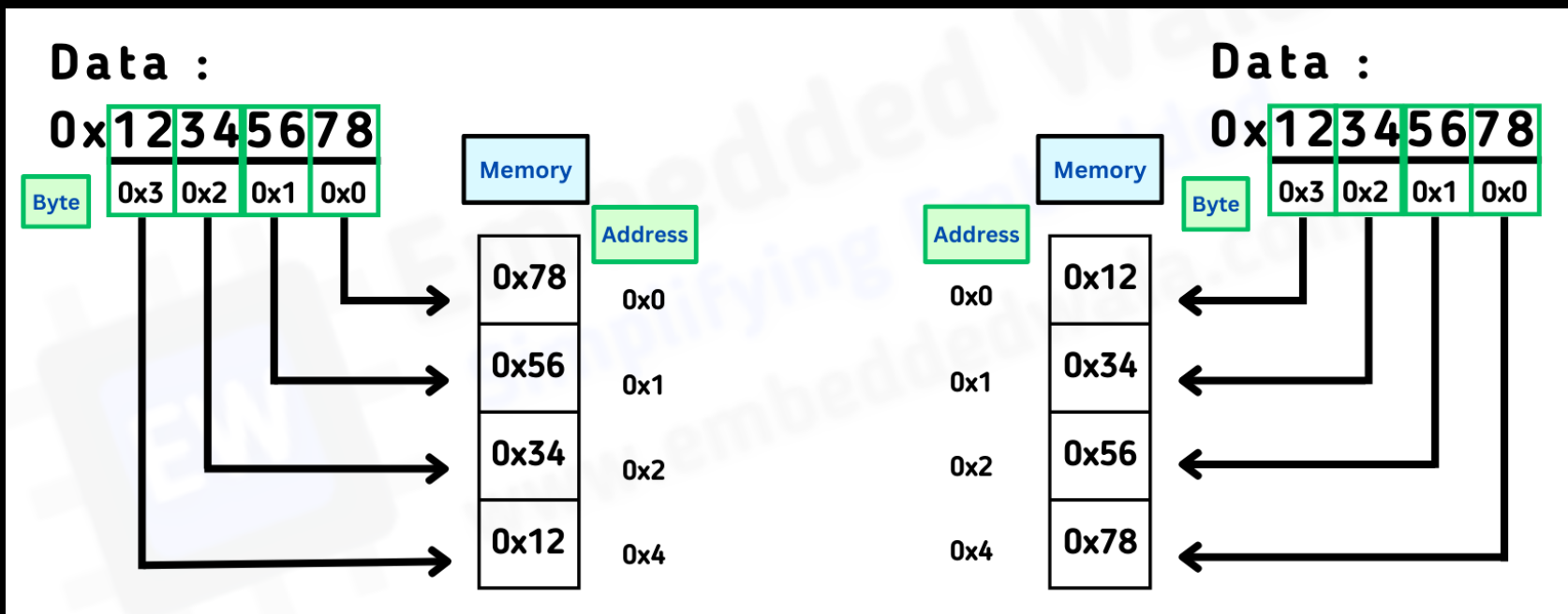
Big-endian

0x12	0x34	0x56	0x78
100	101	102	103

Little-endian

0x78	0x56	0x34	0x12
100	101	102	103

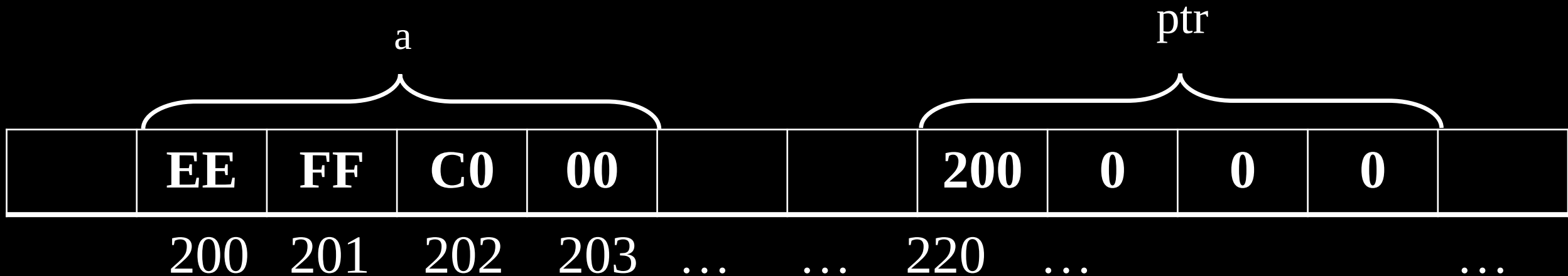
- В современных ПК с архитектурой x86 (Intel, AMD) вы увидите little-endian (прямой порядок байт)
- Сетевые протоколы строились на big endian, называется это сетевым порядком байт



Endianness проблема может возникнуть когда:

- Обмениваетесь данными между двумя системами с разным порядком байт
- Передача данных по сети (TCP/IP)
- Работа с файлами (PNG, JPEG)
- При чтении памяти


```
int a = 0xC0FFEE;  
char *ptr = (char *)&a;  
  
// для little-endian  
printf("first byte of a = %hhx\n", *ptr); // 0xEE  
ptr++;  
printf("second byte of a = %hhx\n", *ptr); // 0xFF  
ptr++;
```



Dynamic array

Динамические массивы - массивы, размер которых можно менять по ходу выполнения программы.

Функции для работы с динамическими массивами (<stdlib.h>):

- malloc()
- calloc()
- realloc()
- free()

malloc

malloc - Memory Alloc - выделяет динамическую память для хранения данных. Имеет 1 аргумент - количество байт, которое необходимо выделить. Возвращает адрес на выделенную память в случае успеха, в случае неудачи (кончилась память, например) - возвращает NULL.

```
int *arr = (int *)malloc(3*sizeof(int));

if (arr == NULL) {
    printf("Error malloc!\n");
    return -1;
}
```

malloc

```
int *arr = (int *)malloc(3*sizeof(int));  
  
if (arr == NULL) {  
    printf("Error malloc!\n");  
    return -1;  
}
```



malloc

```
arr[0] = 123; arr[1] = 2; arr[2] = 3;
```



free

`free(arr);`

arr



800						55555				2				3				
100	101	102	103	...		800	801	802	803	804	805	806	807	808	809	810	811	

`arr = NULL;`

arr

0						55555				321				0xfff12414				
100	101	102	103	...		800	801	802	803	804	805	806	807	808	809	810	811	

Memory leak

Утечка памяти - ситуация, когда программа выделяет память и не освобождает, например:

```
int *ptr = malloc(10000);  
ptr = malloc(20000);
```

В случае если ваша програма завершилась, а вы не сделали free - то, **ОС подчистит всю память за вас**. Однако если ваша программа работает долго в цикле и в ней постоянно выделяется память и не освобождается..

В языке Си нет сборщика мусора

calloc

calloc делает тоже самое что и **malloc**, но инициализирует данные нулями. Первый аргумент – количество элементов, второй количество байт для каждого элемента

```
int *arr = (int *)calloc(3, sizeof(int));  
// или (int *)calloc(1, 3 * sizeof(int));
```


С помощью malloc и memset можно добиться эффекта calloc

```
// 3 элемента по sizeof(int)
int *arr = (int *)malloc(3 * sizeof(int));

// начиная с адреса arr установить 12 нулей
memset(arr, 0, 3 * sizeof(int));
```

realloc

```
char *str = (char *)malloc(N * sizeof(char));  
if (str == NULL) return -1;  
// ..  
// уменьшим массив до M байт  
str = realloc(str, M * sizeof(char));  
if (str == NULL) return -1;
```

realloc позволяет перевыделять память, сохраняя при этом оригинальные значения в выделенной памяти.

// Двумерный массив (матрица)

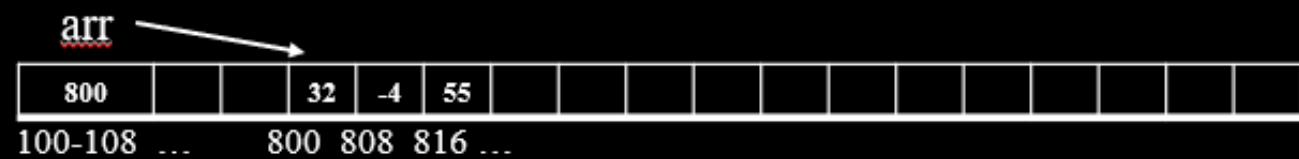
```
#define N 3
```

```
char **arr = (char **)malloc(N * sizeof(char *));
```

```
for (int i = 0; i < N; i++) {  
    arr[i] = (char *)malloc(N * sizeof(char));  
    arr[i] = 'A' + i;  
}
```

```
for (int i = 0; i < N; i++) {  
    free(arr[i]);  
    arr2[i] = NULL;  
}
```

```
free(arr);  
arr2 = NULL;
```



Variable scope

Глобальная область

```
int a;
```

```
int func(void) {  
    static int b;  
    // ..  
}
```

Variable scope

Глобальная область

```
int a;
```

```
int func(void) {  
    static int b;  
    // ..  
}
```

Variable scope

Блок

```
int func(int a) {  
    // ..  
    {  
        int b;  
    }  
    // ..  
}
```

```
#include <stdio.h>

int a = 15;

void func(void) {
    printf("global a = %d\n", a);
}

int main(void) {
    int a = 10;
    {
        int a = 20;
    }
    printf("main a = %d\n", a);
    func();
}
```

Модель памяти языка Си

Stack vs Heap

Virtual and Physical memory

Github

<https://github.com/kruffka/C-Programming>

